

PERTEMUAN KE – 13

Object Oriented Programming

A. TUJUAN

- Memahami dasar OOP PHP

B. TEORI SINGKAT

Bisa dibayangkan alam semesta kita terbuat dari objek yang berbeda seperti matahari, bumi, bulan dll. Demikian pula kita bisa membayangkan mobil kami terbuat dari objek yang berbeda seperti roda, kemudi, gigi dll. Caranya sama ada berorientasi objek konsep pemrograman yang menganggap segala sesuatu sebagai obyek dan menerapkan perangkat lunak menggunakan objek yang berbeda.

Konsep Berorientasi Obyek:

Sebelum masuk detail, mari mendefinisikan istilah-istilah penting yang berhubungan dengan Object Oriented Programming.

- **class:** Ini adalah programmer-didefinisikan datatype, yang meliputi function lokal serta data lokal. Anda dapat menganggap class sebagai template untuk membuat banyak contoh dari jenis yang sama (atau class) dari objek.
- **Obyek:** Sebuah contoh individu struktur data yang didefinisikan oleh class. Anda mendefinisikan class sekali dan kemudian membuat banyak obyek. Obyek juga dikenal sebagai instance.
- **Anggota variabel:** Ini adalah variabel yang didefinisikan di dalam class. Data ini akan terlihat ke luar class dan dapat diakses melalui function anggota. Variabel ini disebut atribut dari objek setelah objek dibuat.
- **function anggota:** Ini adalah function yang didefinisikan di dalam class dan digunakan untuk mengakses data objek.
- **Inheritance:** Ketika class didefinisikan dengan mewarisi function yang ada dari class Parent maka disebut Inheritance. Di sini Child class akan mewarisi semua atau beberapa function anggota dan variabel dari class Parent.
- **class Parent:** Sebuah class yang diwarisi oleh class lain. Hal ini juga disebut class dasar atau class super.
- **Child class:** Sebuah class yang mewarisi class lain. Hal ini juga disebut subclass atau class turunan.
- **Polimorfisme:** Ini adalah konsep yang berorientasi objek di mana function yang sama dapat digunakan untuk tujuan yang berbeda. Misalnya nama function akan tetap sama tetapi membuat mengambil jumlah yang berbeda argumen dan dapat melakukan tugas yang berbeda.
- **Overloading:** jenis polimorfisme di mana beberapa atau semua operator memiliki implementasi yang berbeda tergantung pada jenis argumen mereka. Demikian pula function juga dapat dipenuhi dengan implementasi yang berbeda.
- **Abstraksi Data:** Setiap representasi data di mana rincian pelaksanaan tersembunyi (diabstraksikan).
- **Encapsulation:** mengacu pada konsep di mana kita merangkum semua function data dan anggota bersama-sama untuk membentuk sebuah objek.
- **Pembuat:** mengacu pada jenis khusus dari function yang akan dipanggil secara otomatis setiap kali ada formasi objek dari class.
- **Destructors:** mengacu pada jenis khusus dari function yang akan dipanggil secara otomatis setiap kali sebuah objek dihapus atau keluar dari ruang lingkup.

Mendefinisikan class PHP:

Bentuk umum untuk mendefinisikan class baru di PHP adalah sebagai berikut:

```
<?php
class phpClass {
    var $var1;
    var $var2 = "string konstanta";
    function myfunc($arg1, $arg2) {
        [..]
    }
}
```

```

        [..]
    }
?>

```

Berikut adalah deskripsi dari setiap baris:

- class bentuk khusus, diikuti dengan nama class yang anda ingin mendefinisikan.
- Satu set kawat gigi melampirkan sejumlah deklarasi variabel dan definisi function.
- Deklarasi variabel mulai dengan bentuk var khusus, yang diikuti oleh nama variabel \$konvensional; mereka juga mungkin memiliki nilai awal untuk nilai konstan.
- Definisi function terlihat seperti function PHP mandiri tetapi lokal untuk class dan akan digunakan untuk mengatur dan data objek akses.

C. PRAKTIK

Praktik 1:

Berikut adalah contoh yang mendefinisikan class tipe Books:

```

<?php
class Books{
    /* Member variables */
    var $price;
    var $title;
    /* Member functions */
    function setPrice($par){
        $this->price = $par;
    }
    function getPrice(){
        echo $this->price . "<br/>";
    }
    function setTitle($par){
        $this->title = $par;
    }
    function getTitle(){
        echo $this->title . " <br/>";
    }
}
?>

```

Variabel \$this adalah variabel khusus dan mengacu pada objek yang sama yaitu. objek itu sendiri.

Membuat Objek dalam PHP

Setelah mendefinisikan class, maka dapat dibuat sebagai banyak benda yang disukai dari jenis class. Berikut adalah contoh cara membuat objek menggunakan operator `new`.

```

$physics = new Books;
$maths = new Books;
$chemistry = new Books;

```

Di sini telah diciptakan tiga obyek dan benda-benda yang independen satu sama lain dan akan memiliki keberadaannya secara terpisah. Selanjutnya akan dilihat bagaimana untuk mengakses function anggota dan anggota proses variabel.

Memanggil function Anggota

Setelah membuat objek, akan dapat dipanggil function-function anggota yang terkait dengan objek. Salah satu function anggota akan dapat memproses variabel anggota obyek terkait saja.

Berikut contoh menunjukkan bagaimana untuk mengatur judul dan harga untuk tiga buku dengan memanggil function anggota.

```

$physics->setTitle( "Physics for High School" );
$chemistry->setTitle( "Advanced Chemistry" );
$maths->setTitle( "Algebra" );

$physics->setPrice( 10 );
$chemistry->setPrice( 15 );
$maths->setPrice( 7 );

```

Sekarang memanggil function-function anggota lain untuk mendapatkan nilai-nilai yang ditetapkan dalam contoh di atas:

```

$physics->getTitle();
$chemistry->getTitle();
$maths->getTitle();
$physics->getPrice();
$chemistry->getPrice();
$maths->getPrice();

```

Ini akan menghasilkan hasil:

```

Physics for High School
Advanced Chemistry
Algebra
10
15
7

```

Praktik 2

function Constructor:

function Constructor adalah jenis khusus dari function yang dipanggil secara otomatis setiap kali sebuah objek dibuat. Jadi kita mengambil keuntungan penuh dari perilaku ini, dengan menginisialisasi banyak hal melalui function constructor.

PHP menyediakan function khusus yang disebut `__construct()` untuk mendefinisikan constructor. Dapat dilewatkan sebanyak argumen yang disukai ke dalam function constructor.

Contoh berikut akan membuat satu constructor untuk class Books dan akan menginisialisasi harga dan judul untuk buku pada saat pembuatan obyek.

```

function __construct( $par1, $par2 ){
    $this->price = $par1;
    $this->title = $par2;
}

```

Sekarang tidak perlu memanggil set function secara terpisah untuk mengatur harga dan judul. Dapat menginisialisasi dua variabel anggota ini pada saat pembuatan obyek. Periksa contoh berikut:

```

$physics = new Books( "Physics for High School", 10 );
$maths = new Books ( "Advanced Chemistry", 15 );
$chemistry = new Books ( "Algebra", 7 );

/* Get those set values */
$physics->getTitle();
$chemistry->getTitle();
$maths->getTitle();

$physics->getPrice();
$chemistry->getPrice();
$maths->getPrice();

```

Hal ini akan menghasilkan hasil sebagai berikut:

Physics for High School
Advanced Chemistry
Algebra
10
15
7

Praktik 3

destructor:

Seperti function constructor, dapat juga ditentukan destructor function menggunakan function `__destruct()`. Dapat dilepaskan semua resources dengan destructor.

Inheritance:

Definisi class PHP opsional dapat mewarisi dari definisi class Parent dengan menggunakan klausa `extends`. Sintaksnya adalah sebagai berikut:

```
class Child extends Parent {  
    <definition body>  
}
```

Pengaruh Inheritance adalah bahwa class Child (atau subclass atau class turunan) memiliki karakteristik sebagai berikut:

- Secara otomatis memiliki semua deklarasi variabel anggota dari class Parent.
- Secara otomatis memiliki semua function anggota sama dengan orang tua, yang (secara default) akan bekerja dengan cara yang sama seperti function-function di parent.

Berikut contoh mewarisi class Buku dan menambahkan function onalitis lebih didasarkan pada kebutuhan.

```
class Novel extends Books{  
    var publisher;  
    function setPublisher($par){  
        $this->publisher = $par;  
    }  
    function getPublisher(){  
        echo $this->publisher. "<br />";  
    }  
}
```

Sekarang selain function turunan, class Novel membuat dua function anggota tambahan.

function Overriding:

Definisi function dalam Child class menimpa definisi dengan nama yang sama di class Parent. Dalam class Child, dapat dimodifikasi definisi dari function Inheritance dari class Parent.

Pada contoh berikut `getPrice` dan `getTitle` function yang override untuk return beberapa nilai.

```
function getPrice(){  
    echo $this->price . "<br/>";  
    return $this->price;  
}  
function getTitle(){  
    echo $this->title . "<br/>";  
    return $this->title;  
}
```

Praktik 4

Public Members:

Kecuali ditentukan beda, properti dan metode dari class bersifat publik. Artinya, dapat diakses dalam tiga kemungkinan situasi:

- Dari luar class di mana ia dinyatakan
- Dari dalam class di mana ia dideklarasikan
- Dari dalam class lain yang implements class di mana ia dinyatakan

Sampai tahap ini, telah dilihat semua anggota sebagai anggota public. Jika ingin dibatasi aksesibilitas anggota class maka didefinisikan anggota class sebagai private atau protected.

Anggota private:

Dengan menunjuk anggota private, Anda membatasi aksesibilitas ke class di mana ia dideklarasikan. Anggota private tidak dapat diakses dari class yang mewarisi class di mana dinyatakan dan tidak dapat diakses dari luar class.

Seorang anggota class dapat dibuat private dengan menggunakan kata kunci private dari anggota.

```
class MyClass {
    private $car = "skoda";
    $driver = "SRK";

    function __construct($par) {
        // Statements here run every time
        // an instance of the class
        // is created.
    }
    function myPublicFunction() {
        return("I'm visible!");
    }
    private function myPrivateFunction() {
        return("I'm not visible outside!");
    }
}
```

Ketika class MyClass diwariskan oleh class lain menggunakan extends, myPublicFunction() akan terlihat, seperti yang akan \$driver. class turunan tidak akan memiliki akses ke myPrivateFunction dan \$mobil, karena mereka dinyatakan private.

Anggota yang protected:

Sebuah properti yang protected atau metode dapat diakses di class di mana dinyatakan, serta dalam class yang menjadi turunan class itu. Anggota yang protected tidak dapat diakses di luar dari dua jenis class. Anggota class dapat dibuat protected dengan menggunakan kata kunci protected dari anggota.

Berikut ini adalah versi yang berbeda dari MyClass:

```
class MyClass {
    protected $car = "skoda";
    $driver = "SRK";

    function __construct($par) {
        // Statements here run every time
        // an instance of the class
        // is created.
    }
    function myPublicFunction() {
        return("I'm visible!");
    }
    protected function myPrivateFunction() {
        return("I'm visible in child class!");
    }
}
```

```
}
```

Praktik 4

interface:

interface didefinisikan untuk memberikan nama function umum untuk pelaksana. Pelaksana yang berbeda dapat menerapkan mereka interface sesuai dengan kebutuhan theri. Anda dapat mengatakan, interface skeltons yang didefinikann oleh pengembang.

Pada PHP5, adalah mungkin untuk mendefinisikan interface, seperti ini:

```
interface Mail {  
    public function sendmail();  
}
```

Kemudian, jika class lain menerapkan interface, seperti ini:

```
class Report implements Mail {  
    // sendMail() Definition goes here  
}
```

Praktik 5

konstanta:

Sebuah konstan agak seperti variabel, dalam hal ini memegang nilai, tapi benar-benar lebih seperti function karena konstan berubah. Setelah Anda mendeklarasikan konstanta, itu tidak berubah.

Mendeklarasikan satu konstan mudah, seperti yang dilakukan di versi ini MyClass:

```
class MyClass {  
    const requiredMargin = 1.7;  
    function __construct($incomingValue) {  
        // Statements here run every time  
        // an instance of the class  
        // is created.  
    }  
}
```

Di class ini, requiredMargin adalah konstanta. Hal ini dinyatakan dengan const kata kunci, dan dalam kondisi itu dapat berubah menjadi apa pun selain 1.7. Perhatikan bahwa nama konstanta ini tidak memiliki \$terkemuka, seperti nama variabel yang dapat dilakukan.

Praktik 6

class Abstrak:

Sebuah class abstrak adalah salah satu yang tidak bisa dipakai, hanya mewarisi. Anda mendeklarasikan sebuah class abstrak dengan kata kunci abstrak, seperti ini:

Ketika mewarisi dari class abstrak, semua metode ditandai abstrak di class deklarasi parent harus didefinisikan oleh Child; selain itu, metode ini harus didefinisikan dengan visibillity yang sama.

```
abstract class MyAbstractClass {  
    abstract function myAbstractFunction() {  
    }  
}
```

Perhatikan bahwa definisi function di dalam class abstrak juga harus didahului dengan kata kunci abstrak. Hal ini tidak legal untuk memiliki definisi function abstrak dalam class non-abstrak.

Keyword Static:

Mendeklarasikan anggota class atau metode sebagai statis membuat mereka dapat diakses tanpa perlu Instansiasi class. Seorang anggota dinyatakan sebagai statis tidak dapat diakses dengan objek class instantiated(meskipun metode statis bisa).

Coba contoh berikut:

```
<?php
class Foo
{
    public static $my_static = 'foo';

    public function staticValue() {
        return self::$my_static;
    }
}
print Foo::$my_static . "\n";
$foo = new Foo();
print $foo->staticValue() . "\n"
```

Praktik 7

Final Keyword:

PHP 5 memperkenalkan kata kunci final, yang mencegah Child class dari override metode dengan awalan definisi dengan final. Jika class itu sendiri didefinisikan final maka tidak dapat diextends.

Berikut hasil contoh di Fatal error: Tidak dapat mengganti metode BaseClass final::moreTesting()

```
<?php
class BaseClass {
    public function test() {
        echo "BaseClass::test() called<br>";
    }

    final public function moreTesting() {
        echo "BaseClass::moreTesting() called<br>";
    }
}

class ChildClass extends BaseClass {
    public function moreTesting() {
        echo "ChildClass::moreTesting() called<br>";
    }
}
?>
```

Praktik 8

Memanggil constructor Parent:

Alih-alih menulis constructor yang sama sekali baru untuk subclass, mari kita menulis dengan memanggil constructor orang tua secara eksplisit dan kemudian melakukan apa pun yang diperlukan selain untuk Instansiasi subclass. Berikut adalah contoh sederhana:

```
class Name
{
    var $_firstName;
    var $_lastName;
    function Name($first_name, $last_name)
    {
        $this->_firstName = $first_name;
        $this->_lastName = $last_name;
    }
    function toString() {
        return($this->_lastName . ", " . $this->_firstName);
    }
}
```

```

    }
}
class NameSub1 extends Name
{
    var $_middleInitial;
    function NameSub1($first_name, $middle_initial, $last_name) {
        Name::Name($first_name, $last_name);
        $this->_middleInitial = $middle_initial;
    }
    function toString() {
        return(Name::toString() . " " . $this->_middleInitial);
    }
}

```

Dalam contoh ini, kami memiliki class Parent(Name), yang memiliki constructor dua argumen, dan subclass(NameSub1), yang memiliki constructor tiga argumen. constructor function NameSub1 dengan memanggil parent constructor secara eksplisit menggunakan::sintaks(melewati dua argumen di sepanjang) dan kemudian menetapkan lapangan tambahan. Demikian pula, NameSub1 mendefinisikan nya nonconstructor toString() function dalam hal function Parent yang menimpa.

CATATAN: constructor dapat didefinisikan dengan nama yang sama dengan nama class. Ini didefinisikan dalam contoh di atas.

D. LATIHAN

- *Latihan diberikan oleh dosen pengampu pada saat praktikum.*
- *Dikerjakan di laboratorium pada jam praktikum.*

E. TUGAS

- *TBuatlah suatu kelas, yang tidak ada pada panduan praktikum, yang mengandung definisi dasar class, konstruktor dan inheritance*